

■ LEVEL 21 • KNAPSACK PATTERN

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Items + capacity/target + choose or don't choose. Core decision: take the item or skip it.
Reference Examples	0/1 Knapsack • Subset Sum • Partition Equal Subset Sum • Target Sum • Coin Change
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 21 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	0/1 Knapsack	0/1 Knapsack	Absent
2	Partition Equal Subset Sum	Subset Sum	#416
3	Target Sum	Knapsack / Signed Choice	#494
4	Coin Change II	Unbounded Knapsack	#518
5	Last Stone Weight II	Subset Sum / Knapsack	#1049
6	Partition to K Equal Sum Subsets	Backtracking + Subset State	#698
7	Partition Equal Subset Sum	0/1 Knapsack	416
8	Target Sum	Knapsack Counting	494
9	0/1 Knapsack	0/1 Knapsack	■ Absent
10	Coin Change II	Unbounded Knapsack	518

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 0/1 Knapsack

Pattern: 0/1 Knapsack

LeetCode: ■ Absent

■ Input	weights, values, capacity
■ Output	maximum value without exceeding capacity
■ Constraints	each item used at most once
■ Main Idea / Pattern	For each item, choose take or skip. Update capacity right-to-left for 1D DP.
■■ Time Complexity	$O(nC)$
■ Space Complexity	$O(C)$

■ Optimal C++ Code

```
int knapsack(vector<int>& wt, vector<int>& val, int C) {  
    vector<int> dp(C + 1);  
  
    for (int i = 0; i < wt.size(); i++) {  
        for (int c = C; c >= wt[i]; c--)  
            dp[c] = max(dp[c], dp[c - wt[i]] + val[i]);  
    }  
    return dp[C];  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Partition Equal Subset Sum

Pattern: Subset Sum

LeetCode: #416

■ Input	positive integer array
■ Output	whether array can be partitioned into equal sums
■ Constraints	total sum bounded
■ Main Idea / Pattern	Target is total/2. Treat each number as a 0/1 choice.
■■ Time Complexity	O(nS)
■ Space Complexity	O(S)

■ Optimal C++ Code

```
bool canPartition(vector<int>& nums) {  
    int sum = accumulate(nums.begin(), nums.end(), 0);  
    if (sum % 2) return false;  
    int target = sum / 2;  
    vector<bool> dp(target + 1);  
    dp[0] = true;  
    for (int x : nums)  
        for (int s = target; s >= x; s--)  
            dp[s] = dp[s] || dp[s - x];  
    return dp[target];  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Target Sum

Pattern: Knapsack / Signed Choice

LeetCode: #494

■ Input	numbers and target
■ Output	number of + or - assignments producing target
■ Constraints	$n \leq 20$
■ Main Idea / Pattern	Convert signed selection into a subset-sum target when total and target have compatible parity.
■ Time Complexity	$O(nS)$
■ Space Complexity	$O(S)$

■ Optimal C++ Code

```
int findTargetSumWays(vector<int>& nums, int target) {  
    int sum = accumulate(nums.begin(), nums.end(), 0);  
    if (abs(target) > sum || (sum + target) % 2)  
        return 0;  
  
    int need = (sum + target) / 2;  
    vector<int> dp(need + 1);  
    dp[0] = 1;  
  
    for (int x : nums)  
        for (int s = need; s >= x; s--)  
            dp[s] += dp[s - x];  
  
    return dp[need];  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Coin Change II

Pattern: Unbounded Knapsack

LeetCode: #518

■ Input	coins and amount
■ Output	number of combinations to make amount
■ Constraints	coins may be reused
■ Main Idea / Pattern	For each coin, update amounts from low to high so the coin can be reused.
■■ Time Complexity	$O(nA)$
■ Space Complexity	$O(A)$

■ Optimal C++ Code

```
int change(int amount, vector<int>& coins) {  
    vector<int> dp(amount + 1);  
    dp[0] = 1;  
    for (int coin : coins)  
        for (int x = coin; x <= amount; x++)  
            dp[x] += dp[x - coin];  
    return dp[amount];  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Last Stone Weight II

Pattern: Subset Sum / Knapsack

LeetCode: #1049

■ Input	stone weights
■ Output	minimum possible final stone weight
■ Constraints	sum bounded
■ Main Idea / Pattern	Find the largest subset sum $\leq \text{total}/2$. The difference between two groups is minimized.
■ Time Complexity	$O(nS)$
■ Space Complexity	$O(S)$

■ Optimal C++ Code

```
int lastStoneWeightII(vector<int>& stones) {
    int sum = accumulate(stones.begin(), stones.end(), 0);
    int target = sum / 2;
    vector<bool> dp(target + 1);
    dp[0] = true;
    for (int x : stones)
        for (int s = target; s >= x; s--)
            dp[s] = dp[s] || dp[s - x];
    for (int s = target; s >= 0; s--)
        if (dp[s])
            return sum - 2 * s;
    return 0;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Partition to K Equal Sum Subsets

Pattern: Backtracking + Subset State

LeetCode: #698

■ Input	array and k
■ Output	whether array can be partitioned into k equal-sum subsets
■ Constraints	$n \leq 16$
■ Main Idea / Pattern	Each number must belong to exactly one bucket. Backtrack while tracking current bucket sums.
■■ Time Complexity	$O(k^n)$ worst case
■ Space Complexity	$O(n+k)$

■ Optimal C++ Code

```
bool dfs(int i, vector<int>& nums, vector<int>& bucket, int target) {
    if (i == nums.size())
        return true;

    for (int b = 0; b < bucket.size(); b++) {
        if (bucket[b] + nums[i] > target)
            continue;

        bucket[b] += nums[i];

        if (dfs(i + 1, nums, bucket, target))
            return true;

        bucket[b] -= nums[i];

        if (bucket[b] == 0)
            break;
    }

    return false;
}

bool canPartitionKSubsets(vector<int>& nums, int k) {
    int sum = accumulate(nums.begin(), nums.end(), 0);
    if (sum % k) return false;

    sort(nums.rbegin(), nums.rend());
    vector<int> bucket(k);
    return dfs(0, nums, bucket, sum / k);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Partition Equal Subset Sum

Pattern: 0/1 Knapsack

LeetCode: 416

Input	nums
Output	whether array can be split into two equal-sum subsets
Constraints	1 <= nums.size() <= 200; total sum <= 20000
Main Idea / Pattern	Target is total/2; each number can be chosen once, so use descending 0/1 knapsack updates.
Time Complexity	O(n * target)
Space Complexity	O(target)

Optimal C++ Code

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int sum = accumulate(nums.begin(), nums.end(), 0);
        if (sum % 2) return false;
        int target = sum / 2;
        vector<bool> dp(target + 1, false);
        dp[0] = true;
        for (int x : nums) {
            for (int s = target; s >= x; s--) {
                dp[s] = dp[s] || dp[s - x];
            }
        }
        return dp[target];
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Target Sum

Pattern: Knapsack Counting

LeetCode: 494

Input	nums, target
Output	number of + or - assignments reaching target
Constraints	nums.size() <= 20
Main Idea / Pattern	Transform the sign assignment into a subset-sum count when the required transformed target is valid.
Time Complexity	O(n * sum)
Space Complexity	O(sum)

Optimal C++ Code

```
class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        int sum = accumulate(nums.begin(), nums.end(), 0);
        if (abs(target) > sum || (sum + target) % 2) return 0;
        int need = (sum + target) / 2;
        vector<int> dp(need + 1);
        dp[0] = 1;
        for (int x : nums) {
            for (int s = need; s >= x; s--) {
                dp[s] += dp[s - x];
            }
        }
        return dp[need];
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 0/1 Knapsack

Pattern: 0/1 Knapsack

LeetCode: ■ Absent

Input	weights, values, capacity
Output	maximum total value
Constraints	n items; each item used at most once
Main Idea / Pattern	For each item and capacity, choose between skipping and taking the item.
Time Complexity	$O(n * \text{capacity})$
Space Complexity	$O(\text{capacity})$

Optimal C++ Code

```
int knapsack(vector<int>& weight, vector<int>& value, int capacity) {  
    vector<int> dp(capacity + 1, 0);  
    for (int i = 0; i < weight.size(); i++) {  
        for (int cap = capacity; cap >= weight[i]; cap--) {  
            dp[cap] = max(dp[cap],  
                          value[i] + dp[cap - weight[i]]);  
        }  
    }  
    return dp[capacity];  
}
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Coin Change II

Pattern: Unbounded Knapsack

LeetCode: 518

Input	amount, coins
Output	number of combinations that make amount
Constraints	0 <= amount <= 5000
Main Idea / Pattern	Process each coin once as an outer loop and update amounts upward so a coin may be reused.
Time Complexity	O(n * amount)
Space Complexity	O(amount)

Optimal C++ Code

```
class Solution {
public:
    int change(int amount, vector<int>& coins) {
        vector<int> dp(amount + 1);
        dp[0] = 1;
        for (int coin : coins) {
            for (int x = coin; x <= amount; x++) {
                dp[x] += dp[x - coin];
            }
        }
        return dp[amount];
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.